BLOCK 47

Combining Data with Joins

Python 101 • Day 6 • pandas merge(), match logic, and reconciliation

Today's move

from related tables

to one enriched table

without losing or multiplying rows accidentally

BLOCK 47

What this block covers

30 minutes instruction + 30 minutes guided practice

merge()

Use pandas to combine rows from related dataframes.

Join types

Understand inner joins, left joins, and unmatched rows.

Data quality

Use indicators and row counts to find missing reference data.

Join safety

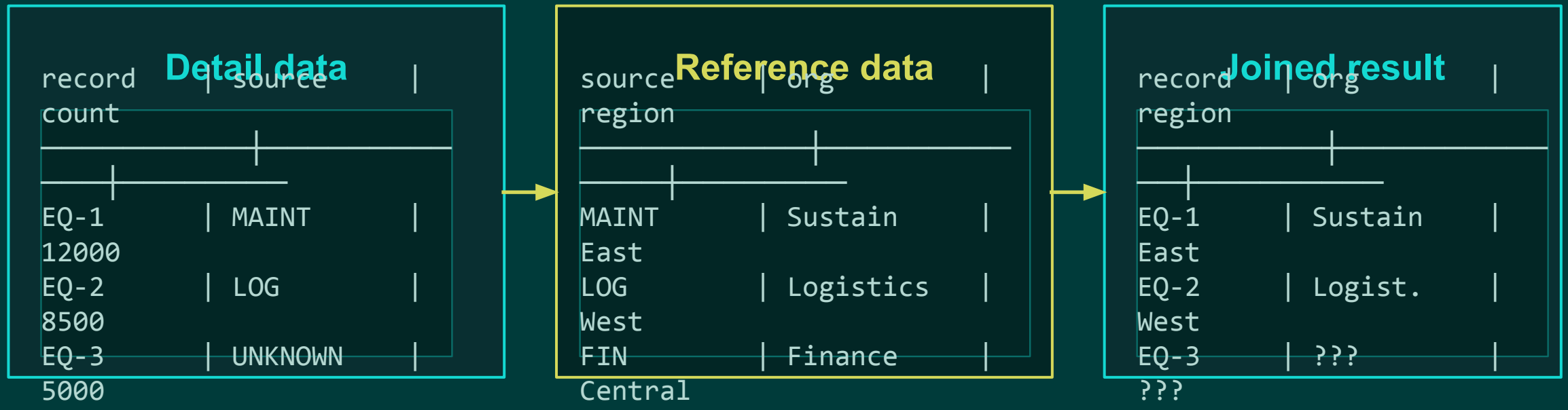
Detect duplicate keys, column collisions, and unexpected expansion.

Main idea: a join is an assumption about how two datasets relate.

BLOCK 47

The goal of a join

Add useful reference fields to detail records using a shared key.



Matching key: source_code

BLOCK 47

A join is not just stacking data

Appending stacks rows. Joining adds columns based on matching keys.

Append / concatenate

Same kind of rows
added underneath existing rows

More rows

Join / merge

Related columns
added beside existing rows

More columns

Ask: Are we adding more observations, or enriching the same observations?

BLOCK 47

Anatomy of merge()

The syntax should express the relationship you intend.

```
joined = equipment.merge(
    source_reference,
    on="source_code",
    how="left"
)
```

equipment
The left dataframe: the population being enriched.

source_reference
The right dataframe: lookup fields to add.

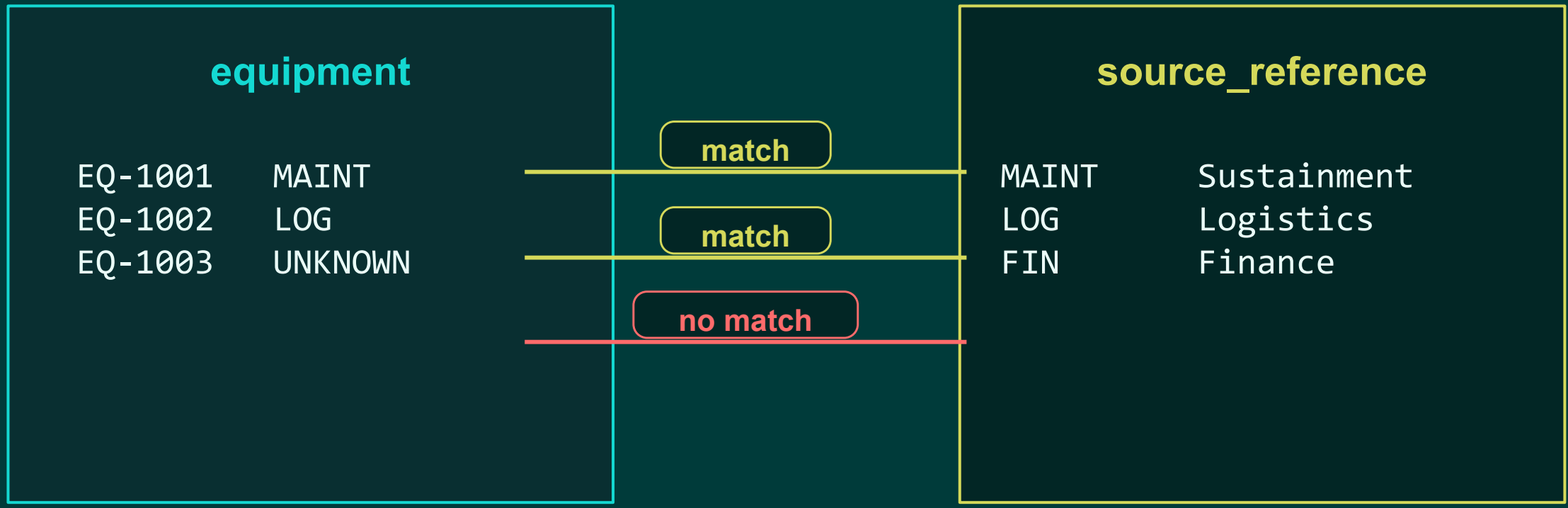
on="source_code"
The key column that appears in both dataframes.

how="left"
The join rule: preserve every left-side row.

BLOCK 47

Join keys are the matching rule

Rows match when the key value is equal on both sides.



The join result depends entirely on the key values.

BLOCK 47

Inner join: keep only matching rows

Useful when unmatched rows should be intentionally excluded.

```
inner_result = equipment.merge(
    source_reference,
    on="source_code",
    how="inner"
)
```

Result keys

MAINT	kept
LOG	kept
UNKNOWN	removed

Danger: unmatched detail records disappear.

BLOCK 47

Inner join can hide data-quality problems

Rows may vanish without obvious errors.

record	source	count
EQ-1	MAINT	12000
EQ-2	LOG	8500
EQ-3	UNKNOWN	5000

record	source	count
EQ-1	MAINT	12000
EQ-2	LOG	8500

Missing from result

EQ-3 still existed in the input.

Before using inner joins, decide whether unmatched rows are allowed to disappear.

BLOCK 47

Left join: preserve the population

Keep every row from the left dataframe and add matches when available.

```
left_result = equipment.merge(
    source_reference,
    on="source_code",
    how="left"
)
```

Result behavior

MAINT	matched
LOG	matched
UNKNOWN	kept with missing fields

Data-quality work often starts with a left join.

BLOCK 47

Choosing inner versus left

Start by naming the population you are responsible for preserving.

Inner join

Use when only successfully related records should remain.

Left join

Use when the left table is the population to keep.

Question 1

Which table contains the rows we cannot lose?

Question 2

What should happen when a reference value is missing?

Join type is a business/data-quality decision, not just a code choice.

BLOCK 47

Use the merge indicator to audit matches

Pandas can label how each row matched.

```
joined = equipment.merge(
    source_reference,
    on="source_code",
    how="left",
    indicator=True
)
```

_merge values

both	matched on both sides
left_only	no reference match
right_only	reference-only row

The indicator turns a join into something you can investigate.

BLOCK 47

Find unmatched records after a left join

Unmatched rows are often the beginning of the investigation.

```
unmatched = joined[
    joined["_merge"] == "left_only"
]

print(unmatched[[
    "record_id",
    "source_code",
]])
```

What to ask

Is the detail code misspelled?
Is the reference table stale?
Is the code valid but missing?

Do not silently drop unmatched records without explaining them.

BLOCK 47

Validate row counts

After a left join with unique reference keys, row count should usually stay the same.

```
input_rows = len(equipment)
output_rows = len(joined)

print(input_rows)
print(output_rows)
```

Expected

For a left join where the reference key is unique:

`input_rows == output_rows`

If rows increase, investigate duplicate keys on the reference side.

BLOCK 47

Duplicate key multiplication

One detail row can become multiple rows if the reference key is duplicated.

record	source
EQ-1	MAINT

source	organization
MAINT	Sustainment
MAINT	Maint
Office	

record	source	organization
EQ-1	MAINT	Sustainment
EQ-1	MAINT	Maint
Office		

This is not a pandas bug. The key matched twice.

BLOCK 47

Check reference-key uniqueness before joining

A supposed lookup key should usually appear once in the lookup table.

```
print(

source_reference["source_code"]
    .is_unique
)
```

```
duplicates = source_reference[
    source_reference.duplicated(
        subset=["source_code"],
        keep=False
    )
]

print(duplicates)
```

Interpretation

True means the key is unique.

False means investigate before trusting row counts.

BLOCK 47

Column-name collisions

If both tables contain the same non-key column, pandas must rename them.

```
record      | source    | status
------------+-----------+-------
EQ-1        | MAINT     | active
```

```
source      | status    | region
------------+-----------+-------
MAINT       | approved  | East
```

```
joined = equipment.merge(
    source_reference,
    on="source_code",
    how="left",
    suffixes=("_record", "_source")
)
```

Bigger lesson

Know which table each field came from.

BLOCK 47

Select only needed reference columns

Join a focused lookup table instead of every column available.

```
reference_columns = source_reference[
    [
        "source_code",
        "organization",
        "region",
    ]
]

joined = equipment.merge(
    reference_columns,
    on="source_code",
    how="left"
)
```

Why this helps

The joined dataframe is easier to read, debug, and explain.

A clean join starts before merge() is called.

BLOCK 47

Join reconciliation checklist

Do not stop at “the code ran.” Verify the relationship behaved as expected.

Before

Identify key columns and check reference-key uniqueness.

During

Choose inner or left based on population preservation.

After

Compare row counts and inspect unmatched rows.

Professional habit

Every join should leave evidence that rows were preserved, matched, or intentionally excluded.

BLOCK 47

Guided exercises: join equipment with source reference

Use clean_equipment_records.csv and source_reference.csv.

Input 1: equipment

```
record_id
source_code
status
record_count
error_count
```

Input 2: source reference

```
source_code
organization
region
manager
```

Goal: create enriched_equipment_records.csv

Exercise 1: perform an inner join

Compare the result row count with the equipment input.

```
inner_result = equipment.merge(
    source_reference,
    on="source_code",
    how="inner"
)

print(len(equipment))
print(len(inner_result))
```

Task

Identify any rows that disappeared and explain why.

Timebox: 5 minutes

Exercise 2: perform a left join with indicator

Preserve equipment rows and label match status.

```
left_result = equipment.merge(
    source_reference,
    on="source_code",
    how="left",
    indicator=True
)

print(len(equipment))
print(len(left_result))
```

Check

Did the left join preserve every equipment record?

Timebox: 5 minutes

BLOCK 47

Exercise 3: list unmatched source codes

Use the _merge indicator to focus the investigation.

```
unmatched = left_result[
    left_result["_merge"] == "left_only"
]

print(
    unmatched["source_code"]
    .drop_duplicates()
)
```

Explain each code

Typo? Missing lookup row?
New legitimate source?

Timebox: 6 minutes

BLOCK 47

Exercise 4: observe duplicate-key multiplication

Intentionally duplicate a reference key, then rerun the left join.

```
dupes = source_reference[
    source_reference.duplicated(
        subset=["source_code"],
        keep=False
    )
]

print(dupes)
```

Observe

What happens to row counts when one reference key appears twice?

Timebox: 6 minutes

BLOCK 47

Exercise 5: create the joined analysis table

Keep only the columns needed for the next analysis step

```
analysis_table = left_result[
    [
        "record_id",
        "source_code",
        "organization",
        "region",
        "status",
        "record_count",
        "error_count",
    ]
]

analysis_table.to_csv(
    "data/output/enriched_equipment_records.csv",
    index=False
)
```

Deliverable

A clean enriched CSV that preserves equipment rows and includes source context.

Timebox: 8 minutes

BLOCK 47

Applied challenge: transactions + organizations

Practice the full join workflow with a different dataset.

Files

```
transactions.csv
organization_reference.csv
```

Identify the join key and preserve every transaction row.

Required evidence

1. Verify reference key uniqueness
2. Perform a left join
3. Identify unmatched org codes
4. Explain possible causes
5. Confirm row count behavior

Success: produce a clean enriched dataframe and a reconciliation note.

Block 47 wrap-up

You can now combine related tables without treating merge() as magic.

Match

Use a clear join key that exists on both sides.

Preserve

Choose inner or left based on which rows matter.

Reconcile

Check row counts, unmatched rows, and duplicate keys.

A join is not merely syntax. It expresses an assumed relationship between datasets.

Next: compare dataframe operations with SQL queries.